

# Your Oracle connection is ordinary

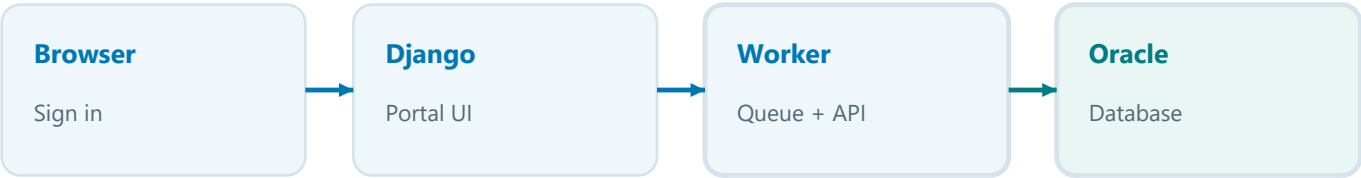
A practical guide to configuration, Windows background operation, office migration and daily controls.

### Oracle does not need this scheduler API token.

The worker connects to Oracle with **ORACLE\_USER**, **ORACLE\_PASSWORD** and **ORACLE\_DSN**. The API token belongs only to the local connection between Django and the worker. It is never sent as your Oracle database password.

The earlier setup extract mixed these two connections without explaining them. This guide separates them and shows exactly which file reads each setting.

### The complete path



Django to worker: local application token. Worker to Oracle: database credentials.

The worker includes the control API automatically. You do not create an API in Oracle, obtain an Oracle token, or start a separate API product.

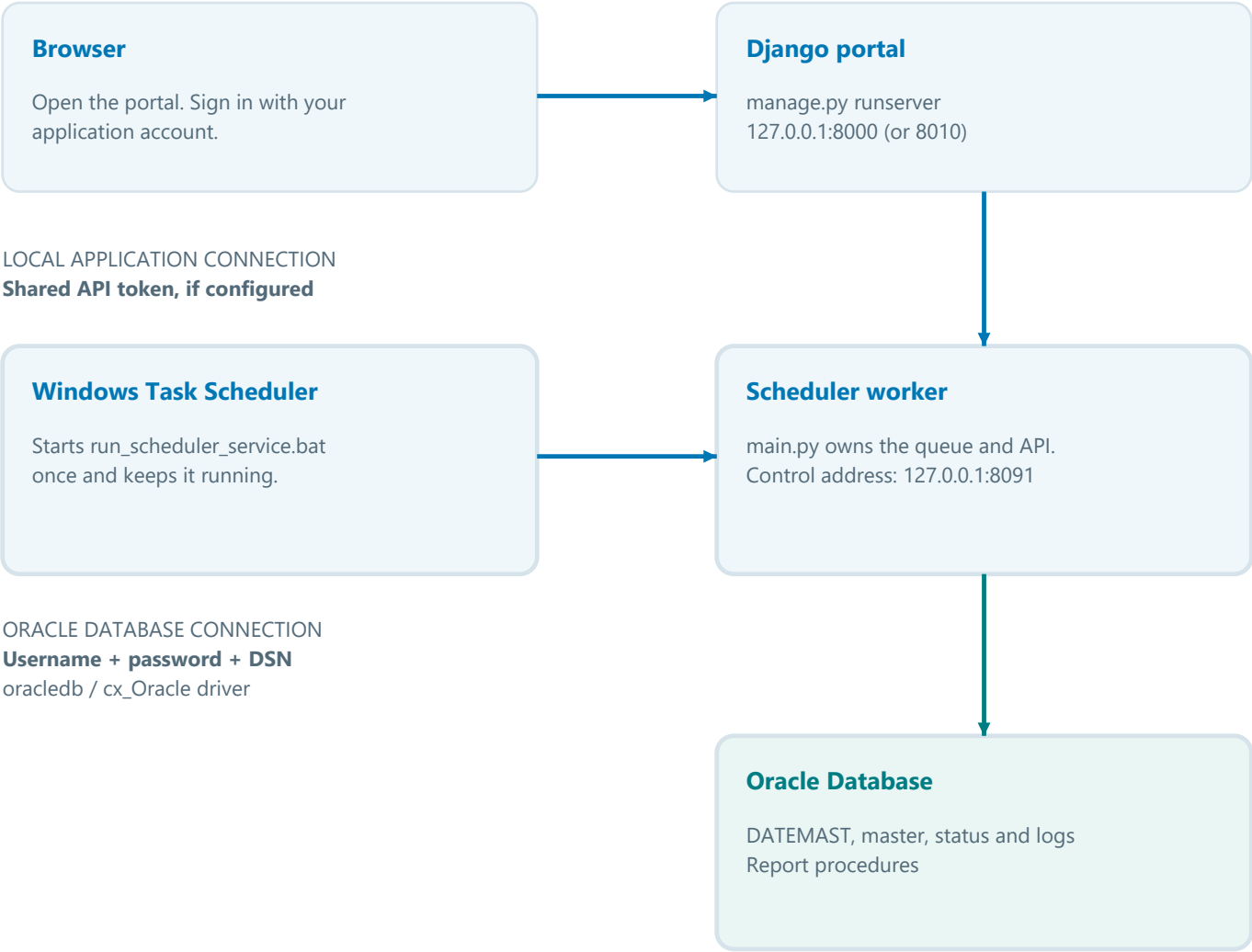
| If you want to...                                      | Read        |
|--------------------------------------------------------|-------------|
| Understand the token, addresses and folder separation  | Pages 2-4   |
| Set up home or office credentials and Python           | Pages 5-9   |
| Run the worker in Windows and open Django separately   | Pages 10-12 |
| Manage schedules, dates, queue, confirmation and leave | Pages 13-16 |
| Understand logging, refreshes and troubleshooting      | Pages 17-19 |
| Find the technical source files and references         | Page 20     |

**Guide scope:** the current project with both folders on one Windows machine. Examples use **D:\ITRP\django\_app**; replace this with your actual folder. Commands marked Command Prompt are for **cmd.exe**. Dates and operating-day examples use IST.

### Worked examples added - pages 21-29

Home and office configuration, Windows startup, a monthly report, calendar dates, retries, leave cover and the complete Django admin model map.

# Two different connections



| Address                             | Meaning                                                                                |
|-------------------------------------|----------------------------------------------------------------------------------------|
| 127.0.0.1:8000<br>or 127.0.0.1:8010 | The web page opened in your browser.                                                   |
| 127.0.0.1:8091                      | The worker control API used by Django. 127.0.0.1 means this same computer.             |
| db-host:1521/service-name           | Example Oracle DSN. The database can be on another server. Its actual port may differ. |

### Office database on another server?

Change the worker's Oracle DSN and credentials. If Django and the worker still run on the same PC, keep their local address at **127.0.0.1:8091**. Do not put the Oracle server address into SCHEDULER\_API\_HOST.

# What each setting really means

| Setting                                                            | Purpose / where it belongs                                                                                       |
|--------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| ORACLE_USER / ORACLE_PASSWORD                                      | Database account credentials. Worker .env only.                                                                  |
| ORACLE_DSN                                                         | Oracle connection string, such as host:1521/service. Worker .env only.                                           |
| SCHEDULER_API_HOST=127.0.0.1                                       | Where the worker listens for local Django requests. Worker .env.                                                 |
| SCHEDULER_API_PORT=8091                                            | Port used by that local control connection. Worker .env.                                                         |
| SCHEDULER_API_BASE_URL                                             | Django destination: http://127.0.0.1:8091. Portal .env.                                                          |
| SCHEDULER_API_TOKEN                                                | One shared application secret in both .env files. Django sends it automatically; the worker checks it.           |
| DJANGO_SECRET_KEY                                                  | Separate Django signing secret for application security. Portal .env only; not a login password.                 |
| SCHEDULER_MASTER_SOURCE=oracle<br>SCHEDULER_CALENDAR_SOURCE=oracle | Select Oracle as the source of schedule definitions and working dates. These are mode switches, not credentials. |

## Who gives you the API token?

Nobody at Oracle. You create a random value once, copy that same value into both applications, and keep it private. You do not type it every day. To generate it in the project Python environment:

```
python -c "import secrets; print(secrets.token_urlsafe(32))"
```

Run the command again to create a **different** value for DJANGO\_SECRET\_KEY. The examples in this guide are placeholders, not working secrets.

## Optional for a local home setup

The current code allows **SCHEDULER\_API\_TOKEN=** to be blank on the loopback address. For the intended blank-token setup, leave it blank in both files. That disables control-request authentication for local callers. A shared random token is the recommended office setup; non-local API binding requires one. This guide keeps the API local.

# Folders, files and responsibilities

|                           |                            |
|---------------------------|----------------------------|
| django_app/               |                            |
| .scheduler-venv/          | Python environment         |
| scheduler_project/        | Worker and Oracle access   |
| .env                      | Oracle + worker settings   |
| main.py                   | Persistent scheduler + API |
| run_scheduler_service.bat |                            |
| control_api.py            |                            |
| database/                 | repositories/              |
| scheduler/                | scheduler_queue/           |
| scheduler_web/            | Django portal              |
| .env                      | Portal + worker address    |
| manage.py                 |                            |
| apps/                     | config/                    |
| templates/                | static/                    |
| start_portal.bat          | Optional combined launcher |

| Part                   | Owns                                                                                                               |
|------------------------|--------------------------------------------------------------------------------------------------------------------|
| Worker                 | DATMAST interpretation, schedule eligibility, execution order, retries, Oracle procedure calls and Oracle logging. |
| Django                 | Pages, users, leave, named confirmation cover, task guides, human checklists and saved portal audit events.        |
| Windows Task Scheduler | Starting and supervising the persistent worker process after Windows starts.                                       |

## Why paths matter

Keep scheduler\_project and scheduler\_web beside each other. The portal setting **SCHEDULER\_PROJECT\_PATH=../scheduler\_project** locates the worker folder. The batch file finds main.py relative to its own location. This is filesystem separation; the running applications exchange commands through the local API.

Moving the parent folder is supported. Update the Windows task's absolute action path and any absolute client or Python paths. Recreate the virtual environment on another machine. Keep runtime database paths stable after setup.

### The two files you run in the requested arrangement

- Background:** scheduler\_project\run\_scheduler\_service.bat
  - Portal:** python manage.py runserver from scheduler\_web
- The other Python modules are loaded by these entry points.

# Step 1 | Prepare Python

## 01 Choose the installation folder

For a new machine, copy source, requirements, migrations, templates, static assets and SQL into the two sibling folders. Example: D:\ITRP\django\_app. Retain backups separately.

## 02 Check the office interpreter

Run **python --version** in the approved environment. This project requires Python 3.10 or newer. The office profile uses Django 5.2. The office cx\_Oracle build, Python and Oracle Client must be compatible; the actual office Python version is still unverified.

Command Prompt - new installation only

```
cd /d D:\ITRP\django_app
python -m venv .scheduler-venv
```

| Home profile                                                                            | Office profile                                                                                              |
|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| ORACLE_DRIVER=oracledb                                                                  | ORACLE_DRIVER=cx_oracle                                                                                     |
| Install the portal and worker home requirements.                                        | Install requirements-office.txt using the approved package source or wheel directory.                       |
| Thin mode needs no Oracle Client. A configured client directory selects Thick mode. [1] | cx_Oracle requires Oracle Client. Use the installed, approved client with matching Python architecture. [2] |

Home - run both commands

```
.scheduler-venv\Scripts\python.exe -m pip install -r scheduler_web\requirements.txt
.scheduler-venv\Scripts\python.exe -m pip install -r scheduler_project\requirements.txt
```

Office - use this profile instead of the home commands

```
.scheduler-venv\Scripts\python.exe -m pip install -r scheduler_web\requirements-office.txt
```

If the office already maintains a compatible interpreter, use it with the required dependencies instead of replacing another program's environment. Set the Windows account environment variable **SCHEDULER\_PYTHON** to its full python.exe path for unattended startup. A value set only in your terminal does not automatically reach the Windows task.

Do not assume the home Python 3.12 installation proves the office cx\_Oracle build will work. Check with the selected office interpreter before starting reports. Django compatibility reference: [3].

## Step 2 | Configure Oracle

Edit **scheduler\_project\env**. Keep working home settings unchanged when preparing an office copy. For a new office installation, start from that folder's **.env.office.example**. Replace the examples below with the real database details.

Office example - use your actual client directory, user, password and DSN

```
ORACLE_DRIVER=cx_oracle
ORACLE_CLIENT_LIB_DIR=C:/Oracle/instantclient_19_XX
ORACLE_USER=YOUR_DATABASE_USER
ORACLE_PASSWORD="YOUR_DATABASE_PASSWORD"
ORACLE_DSN=YOUR_DB_HOST:1521/YOUR_SERVICE_NAME
```

| Value                  | How to fill it                                                                                                                                                  |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ORACLE_DRIVER          | Use oracledb at home or cx_oracle for the office-approved driver. Explicit selection does not silently fall back after an error.                                |
| ORACLE_CLIENT_LIB_DIR  | For cx_Oracle, use the existing client folder, or leave blank if the client already loads through the normal system search. For oracledb, blank uses Thin mode. |
| ORACLE_USER / PASSWORD | Use ordinary database credentials. No custom password encoding is required. Normal dotenv quoting rules still apply.                                            |
| ORACLE_DSN             | Copy the approved Oracle DSN. host:port/service is a service-name example; a SID is a different setting.                                                        |

### Alternative: separate host and service or SID

```
ORACLE_DSN=
ORACLE_HOST=YOUR_DB_HOST
ORACLE_PORT=1521
ORACLE_SERVICE=YOUR_SERVICE_NAME
ORACLE_SID=
```

For a SID-based connection, leave ORACLE\_SERVICE blank and put the SID in ORACLE\_SID. A populated ORACLE\_DSN takes precedence over these fields. Service takes precedence over SID when both are supplied.

#### The actual connection call uses these database values

Conceptually: **driver.connect(user=..., password=..., dsn=...)**.

SCHEDULER\_API\_TOKEN is not an argument to this Oracle connection. The project already contains the connection code. [1, 2]

## Step 3 | Configure the worker

In the **same scheduler\_project\env**, set the source tables, local control address and timing. The service batch explicitly selects Oracle master and calendar mode.

```
SCHEDULER_MASTER_SOURCE=oracle
SCHEDULER_MASTER_TABLE=SCHEDULE_EXTRACT_MASTER
SCHEDULER_MASTER_REFRESH_SECONDS=3600
SCHEDULER_CALENDAR_SOURCE=oracle
SCHEDULER_DATEMAST_TABLE=DATEMAST
SCHEDULER_DATEMAST_DATE_COLUMN=REPORT_DATE
SCHEDULER_CALENDAR_RETRY_SECONDS=300

SCHEDULER_API_HOST=127.0.0.1
SCHEDULER_API_PORT=8091
SCHEDULER_API_TOKEN=PASTE_YOUR_GENERATED_SHARED_VALUE
SCHEDULER_INTERVAL_SECONDS=300
SCHEDULER_MONITOR_INTERVAL_SECONDS=30
SCHEDULER_RETRY_WINDOWS=ALL_DAY
```

Use the real table and column names; schema-qualified names such as BANK.DATEMAST are supported. If the master has TIME\_FLAG and CONFIRMATION columns, set

**SCHEDULER\_MASTER\_OPTIONAL\_COLUMNS=TIME\_FLAG,CONFIRMATION**. Use CONFIRMATION\_NEEDED instead only if that is the actual column name.

### Additional settings for a fresh office installation

```
SCHEDULER_DATA_DIR=runtime/office
SCHEDULER_MASTER_ALLOW_STALE_SNAPSHOT=0
SCHEDULER_ORACLE_LOGGING=0
SCHEDULER_EXTG_TABLE=SCHEDULE_EXTG
SCHEDULER_EXTG_LOG_TABLE=SCHEDULE_EXTG_LOG
SCHEDULER_EXTG_MIRROR=0
```

**Logging=0 is a setup state.** After the office logging schema is ready, set SCHEDULER\_ORACLE\_LOGGING=1 to enable full Oracle status and event delivery. Existing home logging need not be disabled. See page 9.

runtime/office gives the new office worker its own queue database, snapshots and logs. Keeping stale-snapshot fallback disabled prevents an Oracle master failure from silently using a copied home master. Keep these paths stable on later restarts.

DATEMAST successful reads are cached for the local day. The 300-second calendar retry setting applies to failed reads; it does not mean a normal DATEMAST scan every five minutes.

# Step 4 | Configure Django

Edit **scheduler\_web\env**. For a new office installation, start from its **.env.office.example**. This file contains the worker address and portal settings. Oracle credentials stay in the worker folder.

Example for the local office installation described in this guide

```
DJANGO_SECRET_KEY=PASTE_A_DIFFERENT_RANDOM_VALUE
DJANGO_DEBUG=1
DJANGO_ALLOWED_HOSTS=127.0.0.1,localhost
SCHEDULER_PROJECT_PATH=./scheduler_project
DJANGO_DATABASE_PATH=runtime/office/portal.sqlite3

SCHEDULER_API_BASE_URL=http://127.0.0.1:8091
SCHEDULER_API_TOKEN=PASTE_YOUR_GENERATED_SHARED_VALUE
SCHEDULER_API_TIMEOUT_SECONDS=3
SCHEDULER_PORTAL_AUDIT_AUTOSTART=1
SCHEDULER_ALLOW_LOCAL_READ_FALLBACK=0
```

## One value must match exactly

The two **SCHEDULER\_API\_TOKEN** values must match when token checking is enabled. Generate it once, then paste that same value into both files. The worker's port 8091 must match the port in Django's **SCHEDULER\_API\_BASE\_URL**.

| Setting group           | Practical effect                                                                                                                                     |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Django database path    | runtime/office/portal.sqlite3 starts isolated office accounts, leave, guides and audit delivery. Leave blank to retain the existing home db.sqlite3. |
| Audit autostart=1       | Standalone manage.py runserver delivers saved portal audit events to the worker automatically.                                                       |
| Local read fallback=0   | The portal reports an unavailable worker rather than silently presenting local snapshots as a live connection.                                       |
| DEBUG=1 / allowed hosts | These examples are for the local development server on loopback. A shared office web deployment needs the approved production server configuration.  |

## When settings take effect

Restart the worker after changing its **.env**. Restart Django after changing its **.env**. Restart both after changing the shared token. A browser refresh does not reload either process's configuration. Existing Windows/process environment variables take precedence over **.env**; remove stale overrides when moving machines.

# Step 5 | Prepare and verify data

## 01 Confirm the required Oracle objects

The office account needs readable DATEMAST and SCHEDULE\_EXTRACT\_MASTER, master add/change/delete permissions, and EXECUTE permission for the configured report procedures. Procedures must match the project's report-date IN and count OUT contract.

## 02 Prepare Oracle status and event logging

Have the DBA provision or migrate SCHEDULE\_EXTG and SCHEDULE\_EXTG\_LOG using **scheduler\_project\sql\02\_oracle\_logging\_schema.sql**, preserving existing office data. Then set SCHEDULER\_ORACLE\_LOGGING=1. Startup does not create missing grants or rebuild these tables.

### Existing tables are not a startup task

Do not drop SCHEDULE\_EXTG for ordinary startup. The separate rebuild\_oracle\_logging.py utility is for an intentional schema migration. The home schema has previously been integrated; the office schema still needs its own verification.

## 03 Run the connection check with the intended Python

Open Command Prompt in scheduler\_web. This checks the selected Oracle driver, connection and required readable columns. It does not start reports.

```
cd /d D:\ITRP\django_app\scheduler_web
..\scheduler-venv\Scripts\activate
python check_environment.py
```

The checker uses SELECT-only probes, including SELECT 1 FROM DUAL. It does not test report execution, master write permissions or matching API tokens. Logging columns are checked when logging is enabled. A successful check is evidence of Oracle connectivity, not every operational permission.

## 04 Initialize the portal database once

Apply migrations. On a new portal database, create the administrator. Retain existing accounts on a working installation.

```
python manage.py migrate
python manage.py createsuperuser
```

Use an actual Django staff/superuser account for /admin/. A portal SUPERUSER role alone does not grant Django staff access. No default administrator password is supplied.

## Step 6 | Choose how to start

| Entry point                         | What starts / default mode                                                                                             |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| run_scheduler_service.bat           | Persistent worker + API only. <b>Real report execution by default.</b> Windows uses this in the requested arrangement. |
| run_scheduler_service.bat --monitor | Persistent worker + API, but no report procedure calls.                                                                |
| python manage.py runserver          | Django UI + portal audit delivery. Connects to an existing worker; does not start scheduling.                          |
| start_portal.bat                    | Combined worker, Django and audit launcher. Starts a new worker in monitoring mode by default.                         |
| start_portal.bat --execute          | Combined arrangement with report execution enabled for the worker it starts.                                           |
| python main.py                      | Standalone persistent worker. Default enables execution; --monitor disables procedure calls.                           |

### Monitoring still saves changes

Monitoring suppresses report procedure execution. Schedule edits, confirmations, controls and enabled operational logging remain real writes. Use it to inspect connectivity and eligibility with that distinction in mind.

### First worker check on a new installation

Command Prompt - after ensuring another worker is not already running

```
cd /d D:\ITRP\django_app\scheduler_project
run_scheduler_service.bat --monitor
```

Keep the command running while you start Django on page 12. Inspect source, calendar coverage, schedule definitions, confirmation assignments and logging health. Read scheduler\_project\logs\scheduler\_service.log if startup fails.

### Switch to real execution

Stop the monitoring worker after any active work is settled, then start the service batch without --monitor. It cannot switch an existing monitoring process into execution mode. The same distinction applies to the combined launcher.

**Choose one worker arrangement.** Do not run the combined launcher, persistent service batch and repeating run\_scheduler\_cycle.bat together. A one-cycle trigger cannot supply a continuously available portal API.

## Step 7 | Windows background task

Open **Windows Task Scheduler > Create Task**. The task name is your own label. This starts a persistent Python process; it does not register a Windows Service Control Manager service.

| Field              | Set this value                                                                                                                                  |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Name               | ITRP Scheduler Worker                                                                                                                           |
| Account / security | Use the configured office account. Select Run whether user is logged on or not. It needs project/.env read access and runtime/log write access. |
| Trigger            | At startup, with a one-minute delay.                                                                                                            |
| Action             | Start a program                                                                                                                                 |
| Program/script     | C:\Windows\System32\cmd.exe                                                                                                                     |
| Add arguments      | See the exact command below.                                                                                                                    |
| Start in           | D:\ITRP\django_app\scheduler_project<br><b>No surrounding quotes in this field.</b>                                                             |

Add arguments - real execution; replace the installation path

```
/d /c ""D:\ITRP\django_app\scheduler_project\run_scheduler_service.bat""
```

Add arguments - monitoring alternative

```
/d /c ""D:\ITRP\django_app\scheduler_project\run_scheduler_service.bat" --monitor"
```

| Settings / conditions                           | Reason                                                                |
|-------------------------------------------------|-----------------------------------------------------------------------|
| Allow task to be run on demand                  | Lets you start it manually for verification.                          |
| Do not start a new instance                     | Avoids a second instance while one is running. [4]                    |
| Uncheck Stop the task if it runs longer than... | A persistent worker should not stop at the default 72-hour limit. [5] |
| Restart on failure: every 1 minute, 10 attempts | Allows recovery after a failed worker process.                        |
| Review battery and idle conditions              | Keep the machine powered on and awake according to office policy.     |

**Run the task once.** A continuous Running status is expected. No five-minute Windows repetition is needed: the worker has its own cycle. The batch records output in scheduler\_project\logs\scheduler\_service.log.

## Step 8 | Open Django separately

Command Prompt - Windows worker should already be running

```
cd /d D:\ITRP\django_app\scheduler_web
..\scheduler-venv\Scripts\activate
python manage.py runserver
```

Open **http://127.0.0.1:8000/**. Django reads its configured worker address and connects automatically when the UI requests state or sends an action. You do not enter Oracle credentials into the browser.

Optional - retain the earlier portal port 8010

```
python manage.py runserver 127.0.0.1:8010
```

| Check                             | What it proves                                                                                                       |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------|
| http://127.0.0.1:8000/healthz/    | scheduler_connected: true indicates Django can read the worker through the control API.                              |
| http://127.0.0.1:8091/health      | The worker API is alive. This endpoint bypasses the token and is <b>not</b> an Oracle connectivity test.             |
| Source / mode notice and Calendar | Oracle source selected, execution or monitoring mode understood, correct DATEMAST date range and publication cutoff. |
| python check_environment.py       | Separate check of Oracle credentials, selected driver and readable schema; page 9 explains its limits.               |

### Switch from the existing combined launcher

#### 01 Stop future cycles in the portal

Let the current Oracle call finish, then stop the old combined launcher. Keep the existing queue and portal databases.

#### 02 Start the Windows worker task

Then start Django with manage.py runserver. If cycles were paused, re-enable them in the portal; that control persists across restarts.

### Closing Django does not stop this worker

With the Windows task running separately, scheduling continues when the browser or Django is closed. New portal actions require Django. The audit-delivery loop stops with Django and resumes delivery of saved events when Django runs again.

The combined launcher starts its own separate audit-delivery process. Standalone runserver handles it automatically. Other production Django server entry points need a separately supervised **manage.py deliver\_audit\_events --loop**.

# Manage schedules from the UI

## 01 Open Schedules

Sign in with the appropriate SUPERUSER access and open `/scheduler/jobs/`. Use Add schedule, Edit schedule settings or Delete schedule.

| Field group        | What to set                                                                                                 |
|--------------------|-------------------------------------------------------------------------------------------------------------|
| Identity           | Schedule name and the real Oracle procedure/package procedure.                                              |
| Calendar rule      | Frequency, specific dates where needed, report-date margin, SAME_DAY or T+1 behavior, and allowed holidays. |
| Execution controls | Execution window, active state and Maximum automatic attempts.                                              |
| Confirmation       | Whether confirmation is required, primary operator and preferred backup/leave cover.                        |
| Operator guidance  | Expected duration, description and step-by-step guide/checklist.                                            |

## 02 Save and inspect the result

The worker validates the definition and commits it to Oracle `SCHEDULE_EXTRACT_MASTER`. Its local master snapshot refreshes after the commit. Portal-owned guide and assignment information is saved in the portal database and audited.

## 03 Check the planned occurrence

Open Daily tasks for the intended day. Compare the Report date with the Planned execution date and inspect any waiting reason. A saved schedule is not necessarily ready to execute immediately.

### No manual JSON maintenance

In Oracle mode, use the UI for schedule add/change/delete. `Schedule_Master.json` is a worker-managed snapshot. Changes made directly in Oracle are picked up on the next master refresh; they do not require you to edit JSON.

## Editing and deletion

Running definitions cannot be edited or deleted. Deleting a definition retains execution history and audit. Existing IDs are not reused. The Django admin Schedule Profile is operational metadata; master definitions are managed through the Schedules page and worker.

**Expected duration is an estimate.** It helps operators plan the queue. It is not a promise that an Oracle procedure will finish within that duration, and checklist completion does not mean the report has executed successfully.

# Report date, planned day and holidays

**Report date** identifies the business data being processed. **Planned execution date** is the day the occurrence should run. The UI distinguishes them and displays dates as DD/MM/YYYY with operational time in IST.

| Frequency   | Normal period boundary                       |
|-------------|----------------------------------------------|
| Fortnightly | 15th and last calendar day of the month      |
| Monthly     | Last calendar day of the month               |
| Quarterly   | 31 March, 30 June, 30 September, 31 December |
| Half-yearly | 30 September and 31 March                    |
| Annually    | 31 March, the financial-year close           |

These are the period boundaries. DATEMAST availability, configured report-date margin, SAME\_DAY/T+1, permitted holidays and execution windows determine the actual occurrence. T+1 uses the next working execution day under the configured calendar rules.

## DATEMAST publication example

On **10 September**, DATEMAST normally contains working dates only through **9 September**. If the 9th worked, it should be present. A missing 10th is normal and must not be called a holiday just because its row has not yet been published.

Within a successfully loaded historical range through T-1, a present date confirms a working day; a missing date can identify a holiday. An unavailable/empty feed or a date outside the loaded range cannot prove that a day was a holiday.

For today and future days, the calendar uses provisional bank rules: Sundays and second/fourth Saturdays are holidays. Historical DATEMAST presence can record a special working day. Calendar labels explain the source.

## Financial-year rollover remains covered

On 12/09/2026: query 31/03/2025 through 12/09/2026  
On 01/04/2027: query 31/03/2026 through 01/04/2027  
includes 31/03/2027

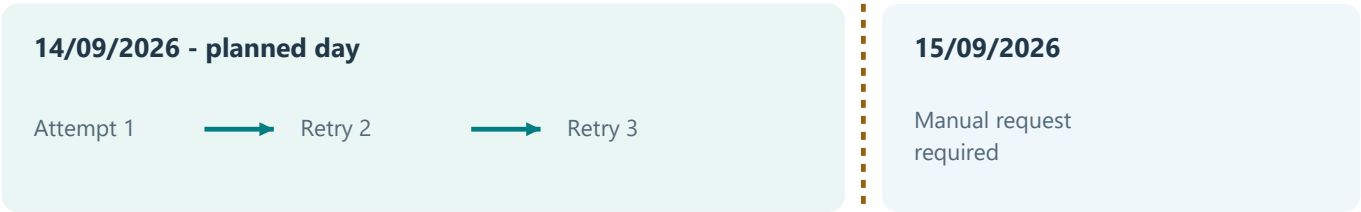
The bounded Oracle query avoids downloading all historical rows. The range rolls forward on 1 April, preserving the immediately preceding 31 March. A SUPERUSER can use Calendar > Refresh from Oracle after a same-day DATEMAST update.

# Queue order and all-day retries

Daily tasks shows what belongs to the selected day. The live queue shows eligible **READY** occurrences in execution order. An empty queue alone does not mean that Oracle is disconnected.

| State / condition                  | What to do                                                                                                            |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Waiting for date, time or DATEMAST | Inspect the reason and the planned day. Correct source data or schedule settings if needed.                           |
| Confirmation needed                | The effective named operator must confirm that occurrence.                                                            |
| READY                              | Eligible for the automatic queue; inspect or change the waiting order.                                                |
| Paused / cancelled                 | Review the operator control and reason before resuming or making a new request.                                       |
| Running                            | Let the current Oracle call finish. Queue changes affect waiting work.                                                |
| Success / failed                   | Inspect execution history, result count and error. Failed work may retry while eligible.                              |
| Manual required                    | The planned execution day has passed, or automatic eligibility is exhausted; inspect and explicitly request recovery. |

## One occurrence, multiple attempts



Maximum automatic attempts is configurable per schedule from **1 to 100**. It counts the first run: 6 means one initial attempt plus up to five retries. With ALL\_DAY, failed work may retry beyond its initial execution window on its planned day, subject to the attempt limit and applicable controls.

After the planned day, each later run requires an explicit manual request. A manual run keeps the original planned date and does not extend automatic retry eligibility.

## Changing the live order

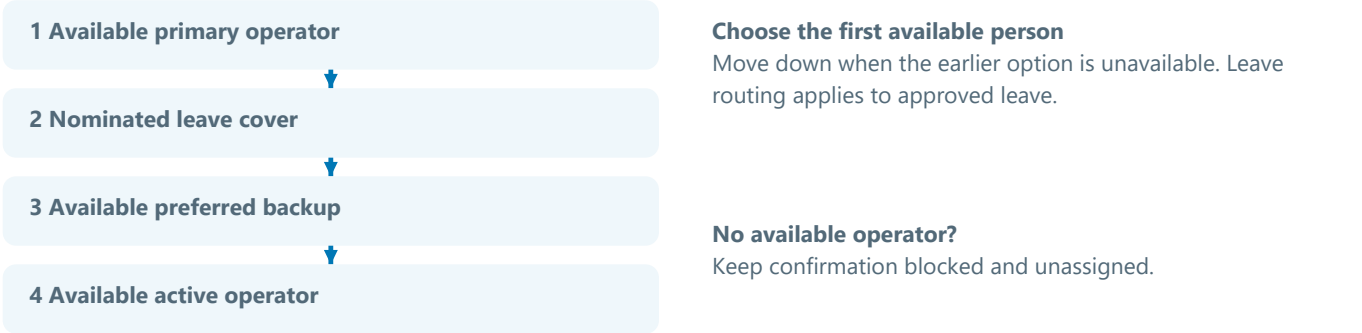
Use Queue to move READY work up/down and save the order, or pause/resume a waiting task. The worker rejects stale queue edits when another operator has changed the revision; reload and review. Ordinary controls are available to active operators; resets, time overrides and service controls require SUPERUSER.

Execution defaults to one selected READY occurrence per five-minute cycle. Reordering does not interrupt a running Oracle procedure.

# Confirmation, leave and admin

A confirmation alert links to unresolved confirmations for the day. Configure who must confirm each schedule; confirmation is attached to the exact report occurrence, not all future runs.

## How the effective confirmer is selected



Only the effective assignee may confirm. The final active-operator fallback is ordered by employee ID. Pending leave does not change availability. Approval rechecks cover, fills available gaps and audits the assignment. Daily fallback does not replace the permanent primary owner.

| Page / admin model                       | Use                                                                                                                                                      |
|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Leave / Leave requests                   | Add or edit pending requests; approve, reject or cancel with the recorded actor.                                                                         |
| Leave schedule coverages                 | Nominate an active, available person for a pending request. Approved/actioned leave is not edited in place; cancel and create a new request for changes. |
| Users, tasks, assignments, delegations   | Manage authorized accounts and human work allocation.                                                                                                    |
| Schedule profiles / runbooks             | Manage operator metadata and guides. Actual master definition CRUD remains on Schedules.                                                                 |
| Audit logs / delivery / runbook progress | Read-only administrative history preserves who did what.                                                                                                 |

Open <http://127.0.0.1:8000/admin/> with an active staff account and the required model permissions. Operational approvals also require active operations-manager access; staff status alone is not enough.

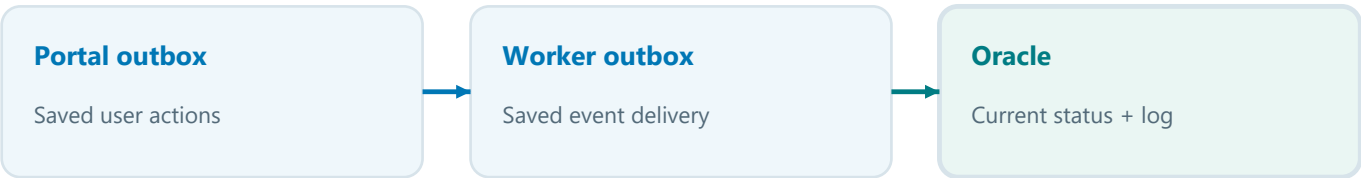
### Human completion and report execution are separate

A checked guide step records the operator's work. Only an actual successful Oracle procedure attempt sets the execution outcome to SUCCESS. Both kinds of activity are logged with their actor and context.

Worked admin examples and all 15 registered models: pages 27-29.

# What is saved and how often

| Data                                  | Authoritative / durable location                                                                           |
|---------------------------------------|------------------------------------------------------------------------------------------------------------|
| Schedule definitions                  | Oracle SCHEDULE_EXTRACT_MASTER in Oracle mode. Local Schedule_Master.json is a refreshed snapshot.         |
| Working report dates                  | Oracle DATEMAST, cached for the local day. Calendar snapshots support the worker's recorded coverage.      |
| Queue and retry state                 | Worker scheduler.db under its configured data directory. Retain this even with Oracle logging enabled.     |
| Current status / event history        | Oracle SCHEDULE_EXTG / SCHEDULE_EXTG_LOG when logging is enabled. Delivery is backed by the worker outbox. |
| Users, leave, guides and portal audit | Portal database: db.sqlite3, or the configured DJANGO_DATABASE_PATH.                                       |



Worker executions and status changes also enter the worker delivery path.

An acknowledgement from the worker means it saved the event durably. The Oracle delivery backlog tells you whether it has reached Oracle. Saved delivery retries survive restarts; keep both local databases.

| Activity               | Default timing / trigger                                                                                                                 |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Schedule evaluation    | Execution mode: 300 seconds. Monitoring: 30 seconds.                                                                                     |
| Master definition read | At startup, then one-hour cache refresh when due. UI master saves refresh immediately after commit.                                      |
| DATEMAST read          | At startup and once per local day; manual Calendar refresh available. Failed reads retry with backoff.                                   |
| Portal audit delivery  | Checks local pending events every 5 seconds while standalone Django runs; sends only pending events.                                     |
| Oracle writes          | Schedule changes, execution, state changes and log delivery occur as needed. They are not restricted to the daily/hourly read intervals. |

A portal screen refresh reads worker state; it does not mean a full DATEMAST or master-table scan. Caching reduces repeated source reads, but it does not eliminate Oracle report and logging traffic. Inspect actual workload before changing timing.

# Find the failing connection

| Symptom                                   | Check / next action                                                                                                                                     |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Browser page does not open                | Start Django. Use 8000 for plain runserver or 8010 if explicitly chosen. Check whether another process owns that port.                                  |
| Page opens, scheduler unavailable         | Check the Windows task is Running, worker log, port 8091 and SCHEDULER_API_BASE_URL. Confirm the protected UI connection, not just /health.             |
| Invalid scheduler API token / 401         | Match SCHEDULER_API_TOKEN in both applications and restart them. This is a Django-to-worker problem; do not change the Oracle password to fix it.       |
| Oracle connection or authentication error | Run check_environment.py with the same interpreter/account as the worker. Review Oracle user/password/DSN, network reachability and readable objects.   |
| Oracle Client cannot load                 | Check selected driver, existing client path and matching architecture under the Windows task account. Restart after configuration changes.              |
| Worker already running; trigger skipped   | Another process owns this installation. Use that worker, or drain and stop it before changing mode. Do not add another repeating trigger.               |
| Existing scheduler is in monitoring mode  | Stop the monitoring process before enabling execution. A new --execute command does not convert it in place.                                            |
| Queue empty                               | Inspect Daily tasks: completed, waiting, confirmation, pause, holiday restrictions, execution window or overdue/manual-required work.                   |
| DATMAST row exists but calendar is stale  | Commit the database change, use Calendar Refresh from Oracle, and check schema/table/date column, loaded range and T-1 cutoff.                          |
| Audit delivery stays pending              | Keep Django audit delivery running; check worker connectivity and Oracle logging health/schema. Retain the outbox databases for retry.                  |
| New .env appears ignored                  | Restart the affected process and check Windows/process overrides. Browser refresh and a second launcher cannot change an existing worker's environment. |

## Two useful log locations

**Worker batch output:** scheduler\_project\logs\scheduler\_service.log

**Worker decisions:** logs\scheduler.log under SCHEDULER\_DATA\_DIR, or under scheduler\_project when no data directory override is set.

# Restart, migrate and retain state

## Normal day

### 01 Check the background task

ITRP Scheduler Worker should be Running. Open Django, sign in, and inspect source/mode, the published report date and the daily task list.

### 02 Resolve waiting work

Review confirmation alerts, leave coverage, queue order and errors. Monitor current work and Oracle audit-delivery backlog.

### 03 End the portal session

Close the browser or stop Django when appropriate. The independently scheduled worker continues while Windows is awake and the task is running.

## Configuration change or planned shutdown

Stop future cycles using the portal and let an active Oracle call finish. Stop the owning worker process/task. Update the relevant .env, run check\_environment.py if database or driver settings changed, and restart. Re-enable scheduler cycles if you paused them. Restart Django too when its settings or the shared token changed.

## Moving to the office

Use the office driver profile and credentials, recreate the virtual environment, verify Oracle schema/grants, and use isolated runtime/office paths for a new installation. Update the Windows task action path. A browser refresh alone cannot migrate a process or change its connection.

| Retain                                    | Why                                                                                             |
|-------------------------------------------|-------------------------------------------------------------------------------------------------|
| Both .env files, held securely            | Deployment settings and credentials. Keep real secrets out of source control and shared guides. |
| Worker and portal databases               | Queue state, accounts, confirmation/leave data and undelivered audit events.                    |
| Snapshots, logs and import/schema backups | Recovery context and diagnosis. Do not treat JSON snapshots as the live master editor.          |
| Migrations, SQL, requirements and tests   | Repeatable setup, schema upgrades and regression checks.                                        |

Back up SQLite consistently: stop writers before copying, or use a database backup method that captures journal/WAL data. Keep home and office state separate.

### Verification boundary

Local Django-to-worker connectivity and launcher tests have passed. Verify the office Python, Oracle credentials, permissions and report procedures on the office machine. This document does not register a Windows task or change running services.

# File reference and further reading

| Path relative to django_app                                                                                | Purpose                                                                        |
|------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| scheduler_project/run_scheduler_service.bat                                                                | Windows task target; continuous worker. --monitor and --help supported.        |
| scheduler_project/main.py<br>scheduler_project/control_api.py                                              | Worker lifecycle and built-in local control API.                               |
| scheduler_project/database/oracle_driver.py                                                                | Driver selection, Oracle Client initialization and normal database connection. |
| scheduler_project/config/settings.py                                                                       | Worker data paths, timing and retry defaults.                                  |
| scheduler_project/repositories/                                                                            | Oracle schedule/calendar persistence and related storage logic.                |
| scheduler_project/sql/02_oracle_logging_schema.sql                                                         | Required current-status and event-log schema for Oracle integration.           |
| scheduler_web/manage.py<br>scheduler_web/check_environment.py                                              | Django commands / SELECT-only Oracle environment checks.                       |
| scheduler_web/apps/scheduler/control_client.py                                                             | Django requests to the worker; sends the application token automatically.      |
| scheduler_web/apps/audit/management/commands/runserver.py                                                  | Standalone runserver audit-delivery lifecycle.                                 |
| scheduler_web/WINDOWS_WORKER_SETUP.md<br>scheduler_web/OFFICE_DEPLOYMENT.md<br>scheduler_web/OPERATIONS.md | Companion editable setup, migration and operations instructions.               |

## Official technical references

[1] [Oracle: normal user/password/DSN connections and Thin/Thick modes](#)

[2] [Oracle: cx\\_Oracle 8.3 installation and Oracle Client requirements](#)

[3] [Django 5.2: supported Python versions](#)

[4] [Microsoft: do not start a new task instance](#)

[5] [Microsoft: default execution limit and unlimited runtime](#)

References checked 12/09/2026. Project behavior was checked against the local implementation and configuration examples. Links above cover external platform behavior; the source files listed here define this application's behavior.

## Continue to the worked examples

Pages 21-29 turn these setup instructions into concrete scenarios with expected results. Pages 28-29 list all application and Django system models available in admin.

# Example 1 | A home connection

These are fictional examples. Replace database names and credentials with your own working values. Nothing here creates an Oracle account or procedure.

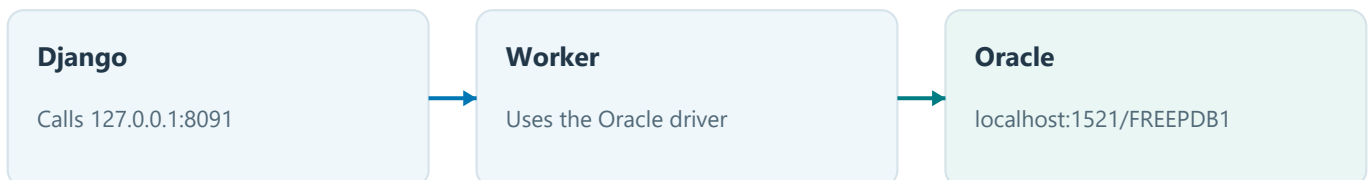
**Situation:** Django and the worker run on your home PC. Assume Oracle listens on localhost:1521 and the database service is FREEPDB1. Your actual service might have another name.

scheduler\_project\.env - local home example with blank control token

```
ORACLE_DRIVER=oracledb
ORACLE_CLIENT_LIB_DIR=
ORACLE_USER=REPORT_SCHEDULER
ORACLE_PASSWORD="EXAMPLE_ONLY_REPLACE_THIS"
ORACLE_DSN=localhost:1521/FREEPDB1
SCHEDULER_MASTER_SOURCE=oracle
SCHEDULER_CALENDAR_SOURCE=oracle
SCHEDULER_API_HOST=127.0.0.1
SCHEDULER_API_PORT=8091
SCHEDULER_API_TOKEN=
```

scheduler\_web\.env - keep your other Django settings from page 8

```
SCHEDULER_API_BASE_URL=http://127.0.0.1:8091
SCHEDULER_API_TOKEN=
SCHEDULER_PORTAL_AUDIT_AUTOSTART=1
```



## Expected result

The worker uses REPORT\_SCHEDULER and its database password to connect to FREEPDB1. The blank API token only disables authentication of local control requests. It does not change Oracle login or remove the Django sign-in screen.

### 01 Test Oracle

From scheduler\_web with the project environment activated, run **python check\_environment.py**. A successful check confirms the Oracle connection and readable schema; it does not test procedure execution.

### 02 Open the portal

Start one worker, then **python manage.py runserver**. Open <http://127.0.0.1:8000/>. This is the browser address; 1521 and 8091 are different connections.

For a token-protected setup, generate a random shared value as shown on page 3, place it in both .env files and restart both processes. The office example follows.

# Example 2 | Move to office Oracle

Assume the approved project folder is D:\ITRP Reports\django\_app and the office already uses a compatible cx\_Oracle and Oracle Client installation.

scheduler\_project\.env - fictitious host/client path; replace before testing

```
ORACLE_DRIVER=cx_oracle
ORACLE_CLIENT_LIB_DIR=C:/Oracle/instantclient_19_22
ORACLE_USER=OFFICE_REPORT_SCHEDULER
ORACLE_PASSWORD="EXAMPLE_ONLY_REPLACE_THIS"
ORACLE_DSN=office-db.example.invalid:1521/REPORTS
SCHEDULER_DATA_DIR=runtime/office
SCHEDULER_MASTER_SOURCE=oracle
SCHEDULER_CALENDAR_SOURCE=oracle
SCHEDULER_MASTER_ALLOW_STALE_SNAPSHOT=0
SCHEDULER_API_HOST=127.0.0.1
SCHEDULER_API_PORT=8091
SCHEDULER_API_TOKEN=PASTE_THE_GENERATED_SHARED_VALUE
```

scheduler\_web\.env - relevant changes; retain remaining settings from page 8

```
DJANGO_SECRET_KEY=PASTE_A_DIFFERENT_RANDOM_VALUE
DJANGO_DATABASE_PATH=runtime/office/portal.sqlite3
SCHEDULER_PROJECT_PATH=../scheduler_project
SCHEDULER_API_BASE_URL=http://127.0.0.1:8091
SCHEDULER_API_TOKEN=PASTE_THE_GENERATED_SHARED_VALUE
SCHEDULER_PORTAL_AUDIT_AUTOSTART=1
```

| Change                    | Why                                                                                    |
|---------------------------|----------------------------------------------------------------------------------------|
| Database DSN and password | The worker now targets the office Oracle database.                                     |
| oracledb to cx_oracle     | Selects the office-approved Python driver. It still uses a normal database connection. |
| runtime/office paths      | Avoids using copied home accounts, queued occurrences or undelivered events.           |
| Keep 127.0.0.1:8091       | Django and the worker are still on the same office PC.                                 |

**Expected result:** check\_environment.py uses the office driver and actual office credentials. No JSON import is needed to read the office master. The example.invalid hostname intentionally cannot be used as the real server.

Provision the logging schema before enabling SCHEDULER\_ORACLE\_LOGGING=1. This configuration cannot create missing tables or grants. Keep an existing system client lookup if it already works; the example client directory is not mandatory.

# Example 3 | Start with Windows

This example uses a folder with a space to show the exact quoting. First stop any existing worker/combined launcher after active work finishes.

| Task Scheduler field | Example                                                                          |
|----------------------|----------------------------------------------------------------------------------|
| Name                 | ITRP Scheduler Worker                                                            |
| Trigger              | At startup; delay one minute                                                     |
| Program/script       | C:\Windows\System32\cmd.exe                                                      |
| Start in             | D:\ITRP Reports\django_app\scheduler_project<br>Do not put quotes in this field. |
| Already running      | Do not start a new instance                                                      |

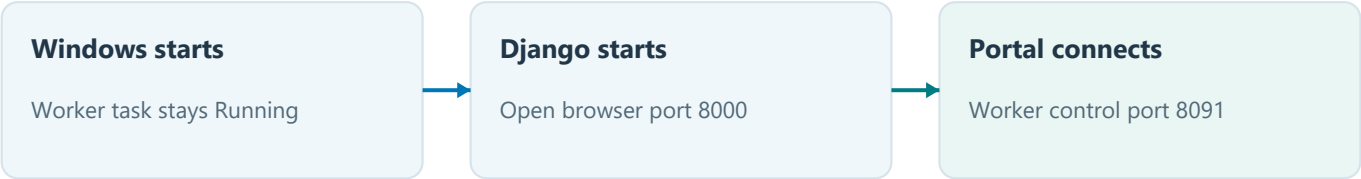
Add arguments - enables real execution

```
/d /c ""D:\ITRP Reports\django_app\scheduler_project\run_scheduler_service.bat""
```

For monitoring, add **--monitor** using the exact quoting on page 11. Apply its unlimited runtime and restart-on-failure settings.

Then start Django from Command Prompt

```
cd /d "D:\ITRP Reports\django_app\scheduler_web"
..\scheduler-venv\Scripts\activate
python manage.py runserver
```



| What you do next                                                                 | Expected observation                                                                                        |
|----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Open <a href="http://127.0.0.1:8000/healthz/">http://127.0.0.1:8000/healthz/</a> | scheduler_connected: true when Django can read the worker.                                                  |
| Close only the browser                                                           | The worker and Django continue running.                                                                     |
| Stop only Django                                                                 | The separate worker continues scheduling. Start Django again to use the UI and forward saved portal events. |
| Stop the Windows worker task                                                     | Automatic processing stops; an open portal reports the worker unavailable.                                  |

**No repeating Windows trigger is needed.** The worker has its own cycle. Stop any old interactive worker first; do not run the legacy one-cycle batch beside it.

# Example 4 | Add a monthly report

Illustration only: the DBA has supplied a real compatible procedure. PKG\_REPORTS.MONTH\_END below is a fictional name, not a procedure created by this guide.

## 01 Open Schedules > Add schedule

Use an authorized SUPERUSER account. Enter the example below, substituting the real procedure and actual operators.

| Form field                    | Example value                                                                |
|-------------------------------|------------------------------------------------------------------------------|
| Schedule ID                   | 9001, only if that ID has never been used                                    |
| Name / procedure              | MONTH_END_RECON / PKG_REPORTS.MONTH_END                                      |
| Run frequency                 | Monthly                                                                      |
| DATEMAST margin               | T - keep the base report date unchanged                                      |
| Execute on the occurrence day | Unchecked for T+1 behavior                                                   |
| Execution window              | 10:00 to 18:00                                                               |
| Holiday permissions           | None for this example                                                        |
| Maximum automatic attempts    | 6 - first attempt plus up to five retries                                    |
| Confirmation / ownership      | Required; primary Asha (E101), preferred backup Ravi (E102)                  |
| Expected duration             | 20 minutes                                                                   |
| Guide                         | One step per line: check report date; verify input; record result            |
| Active / operator reason      | Keep inactive while checking; reason: Create monthly reconciliation schedule |

## 02 Save and verify persistence

The worker commits the master definition to SCHEDULE\_EXTRACT\_MASTER and refreshes its master snapshot. Guide/owner context is saved by Django. Check the saved record and Daily tasks instead of editing Schedule\_Master.json.

### Example dates and assumptions

Once active, assume 30/09/2026 and 01/10/2026 are working days with a healthy DATEMAST feed. Margin T and next-working-day execution give **Report 30/09/2026** and **Planned 01/10/2026**. Time, confirmation and other controls still determine readiness.

The procedure must accept report-date IN and count OUT parameters. Save does not test execution. With margin T+1 instead, the report-date parameter would be 01/10/2026; the next-working-day execution setting is separate.

# Example 5 | Read DATEMAST correctly

Assume today is 10/09/2026, Oracle loaded the covered range successfully, and DATEMAST is complete through T-1. These rows are illustrative.

| Date / row           | Calendar interpretation                                                                          |
|----------------------|--------------------------------------------------------------------------------------------------|
| 05/09/2026 - present | Working day. It is the first Saturday; historical presence confirms it.                          |
| 06/09/2026 - absent  | Sunday. Absence is consistent with the bank closure.                                             |
| 07/09/2026 - absent  | A historical missing date within the successfully loaded complete range is treated as a holiday. |
| 09/09/2026 - present | Latest published working report date in this example.                                            |
| 10/09/2026 - absent  | Today. Absence is expected for a T-1 table and is not evidence of a holiday.                     |
| 12/09/2026 - future  | Second Saturday: a provisional bank holiday under the default rules.                             |

## You inserted a missing working date today

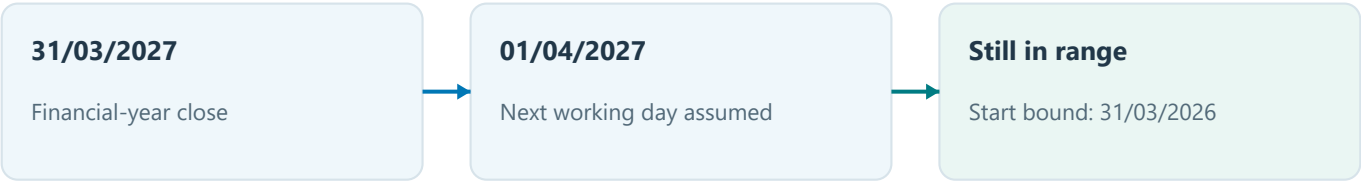
Suppose 07/09 was actually working and you correct and commit its row in Oracle. Open **Calendar > Refresh from Oracle** as SUPERUSER. After the refresh succeeds, it should show as working. No manual JSON edit is needed. If it does not, verify the configured schema/table/date column and source notice.

## Margin changes the procedure date by calendar days

For a DAILY task on Monday 14/09, assume DATEMAST's previous working date is Friday 11/09. The second Saturday and Sunday are closed. Next-working-day execution is selected.

| Margin | Procedure report date | Planned day |
|--------|-----------------------|-------------|
| T      | 11/09/2026 (Friday)   | 14/09/2026  |
| T+1    | 12/09/2026 (Saturday) | 14/09/2026  |

## Financial-year rollover example



For annual frequency with margin T and next-working-day execution, assume 1 April is working. The report uses 31/03/2027: the new bounded query includes it. Historical execution records are retained separately.

# Example 6 | Retry and reorder work

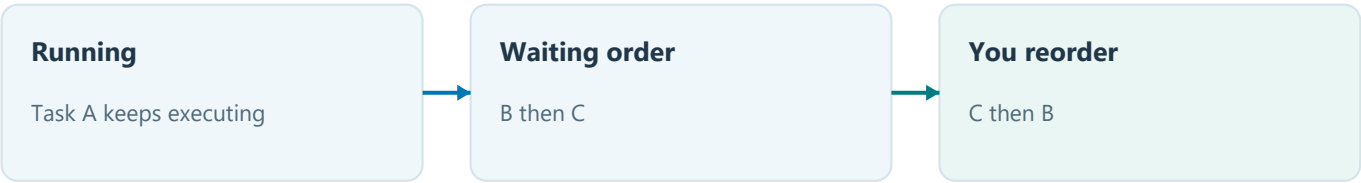
All times below are IST. A retry remains subject to the configured attempt limit and normal confirmation/control checks.

**Situation:** a task is planned for 14/09/2026. Its first window is 10:00-18:00, maximum automatic attempts is 6, and SCHEDULER\_RETRY\_WINDOWS=ALL\_DAY.

| Time               | Event               | Expected behavior                                                                |
|--------------------|---------------------|----------------------------------------------------------------------------------|
| 14/09 10:00        | Attempt 1 fails     | The first run counts toward the six total automatic attempts.                    |
| 14/09 10:05        | Attempt 2 fails     | May retry again that day when selected by the worker.                            |
| 14/09 18:30        | Failed work remains | ALL_DAY can allow another retry after the initial window, while attempts remain. |
| 14/09, attempt 6   | Still fails         | Automatic attempt limit reached. It does not create an attempt 7 automatically.  |
| 15/09 00:00 onward | Planned day passed  | No automatic catch-up. Inspect and explicitly request a manual run.              |

Times illustrate eligibility, not a guaranteed dispatch timetable. Other queued work, paused controls or a running procedure can delay selection. A manual attempt does not reset the planned date or reopen automatic retries on 15 September.

## Queue example while another procedure runs



### 01 Move Task C above Task B and save

The waiting order changes. Task A continues; reordering is not an Oracle procedure interrupt.

### 02 Pause Task C

It waits until resumed. Eligible Task B can be selected on a later cycle, subject to its own controls.

### 03 Another operator changes the queue first

If your saved revision is stale, the worker rejects the change. Refresh the queue, review the current order and save again.

## Expected audit evidence

Queue moves, pause/resume actions and execution attempts retain the actor and context in the operational audit. A manual request is a new explicit action; it should not be simulated by editing stored success or retry rows in admin.

# Example 7 | Leave and confirmation

Use actual users and the real schedule ID in your installation. Names and IDs here are fictional training examples.

| Person                | Assigned role in this example                          |
|-----------------------|--------------------------------------------------------|
| Asha - employee E101  | Primary confirmer for MONTH_END_RECON                  |
| Ravi - employee E102  | Preferred backup                                       |
| Meera - employee E103 | Available nominated cover during Asha's approved leave |

### 01 Create a pending leave request

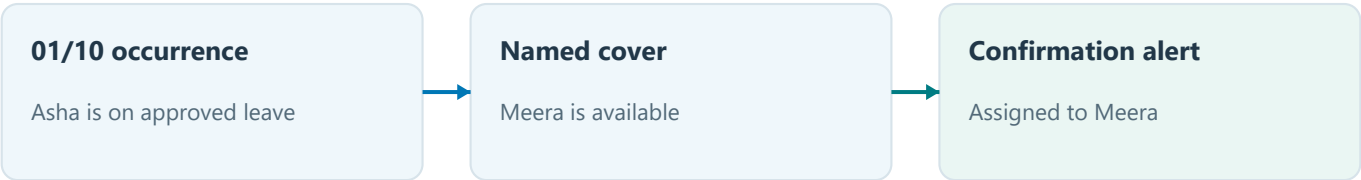
In Django admin > Leave requests > Add, select Asha, start 01/10/2026, end 02/10/2026 and a reason. Save. Pending leave does not yet change availability.

### 02 Nominate Meera as cover

In Leave schedule coverages > Add, select that pending request, the actual MONTH\_END\_RECON schedule ID and Meera. Save. The schedule must require confirmation and belong to Asha; Meera must be available for the leave period.

### 03 Approve through the action menu

As an authorized operations manager with the Django permissions, select the request and choose Approve selected pending requests. Approval rechecks availability and coverage.



| Variation                       | Expected result                                                                                                                |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Meera is available              | Meera confirms this occurrence. Ravi remains the preferred backup on the schedule profile.                                     |
| No nominated cover is available | The workflow checks preferred backup and then other available active operators.                                                |
| No suitable operator exists     | Required coverage blocks leave approval; an unresolved execution confirmation remains blocked rather than being auto-approved. |
| The approved leave dates change | Cancel the existing request and create a new one. Approved/actioned dates are not edited directly.                             |

The confirmation applies to this exact report occurrence. The permanent primary owner remains Asha; the actual confirmer and assignment source are recorded. Completing a runbook checkbox alone does not approve or execute the report.

# Example 8 | Manage Django records

All ten application models are registered. Availability of Add, Change and workflow actions depends on Django permissions and the project's operational roles.

Only if you need a new Django administrator; existing users can be retained

```
cd /d "D:\ITRP Reports\django_app\scheduler_web"
..\scheduler-venv\Scripts\activate
python manage.py createsuperuser
```

Open **http://127.0.0.1:8000/admin/** (or your selected portal port). Use an active staff account. An operational manager also needs active operations access; the portal SUPERUSER role and Django staff permission are distinct.

| Application model     | What you can manage / inspect                                                                                                   |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------|
| AdminUser             | Accounts, employee ID, display name, role, active/staff status and authorized permissions.                                      |
| Task                  | Human task name, description, owner and active state.                                                                           |
| TaskAssignment        | Assign an operator to a human task and mark a primary assignment.                                                               |
| TaskDelegation        | Temporary from/to assignment with dates and reason. Same-person and reversed-date submissions are rejected.                     |
| LeaveRequest          | Pending requests and validated approve/reject/cancel actions. Actioned history is retained.                                     |
| LeaveScheduleCoverage | Available named cover for a confirmation-required schedule during pending leave.                                                |
| ScheduleProfile       | Existing schedule ownership, preferred backup, expected duration and guide. Create the master schedule through Schedules first. |
| RunbookProgress       | Inspect report-specific human check completion; read-only history.                                                              |
| AuditLog              | Inspect actor, action, before/after detail and reason; read-only.                                                               |
| AuditDelivery         | Inspect pending/accepted delivery, attempts and errors; read-only.                                                              |

### Human Task is not the Oracle master

Adding a Task in Django admin does not create an Oracle schedule or procedure. Use **Schedules > Add schedule** for SCHEDULE\_EXTRACT\_MASTER. From an existing Schedule Profile, use its link to the operations portal for master settings and execution controls.

For an existing Schedule Profile, an example operational\_steps value is ["**Check report date**", "**Verify input**", "**Record result**"]. It must be a JSON list of up to 20 non-empty text steps. Expected duration is an estimate, not a procedure timeout.

# Example 9 | Permissions and audit

The four previously missing Django system models are now registered as read-only. Auth Group remains manageable and its changes use the durable application audit path.

| Django system model | Admin behavior                                                                                        |
|---------------------|-------------------------------------------------------------------------------------------------------|
| Group               | Create/edit groups and assign permissions. Membership is managed on users. Group changes are audited. |
| Permission          | Read-only inspection of available permissions; assign them through groups or users.                   |
| ContentType         | Read-only inspection of Django's model catalogue.                                                     |
| LogEntry            | Read-only Django admin action history, alongside the application AuditLog.                            |
| Session             | Read-only session overview with expiry/status. Session keys and stored session contents are hidden.   |

## Worked example: give a staff operator a narrow admin view

### 01 Create an appropriate group

As an authorized superuser, open Groups > Add, name it Leave viewers, and select the view permission for Leave request. Save.

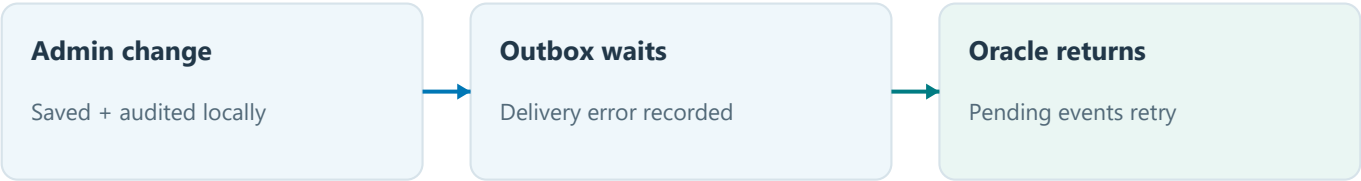
### 02 Add the user to that group

Open Users, assign the group and enable staff status for the user who should enter admin. Keep the user active. Do not grant superuser status just to make a view appear.

### 03 Sign in as that user

The user sees permitted admin models. Project rules still restrict ordinary operators to their own leave records. Approval requires both the relevant Django permission and an operations-manager role.

## Worked example: Oracle temporarily unavailable



AuditLog records the actor and change; AuditDelivery records worker acceptance. Check the worker's backlog for Oracle delivery. This preserves audit events during outages. Master writes and report procedures still require Oracle to be available.

### Expected recovery

Pending delivery retries when connectivity returns. Keep Django delivery running and retain both databases. Inspect the backlog; do not rewrite acceptance fields or audit history to clear it.